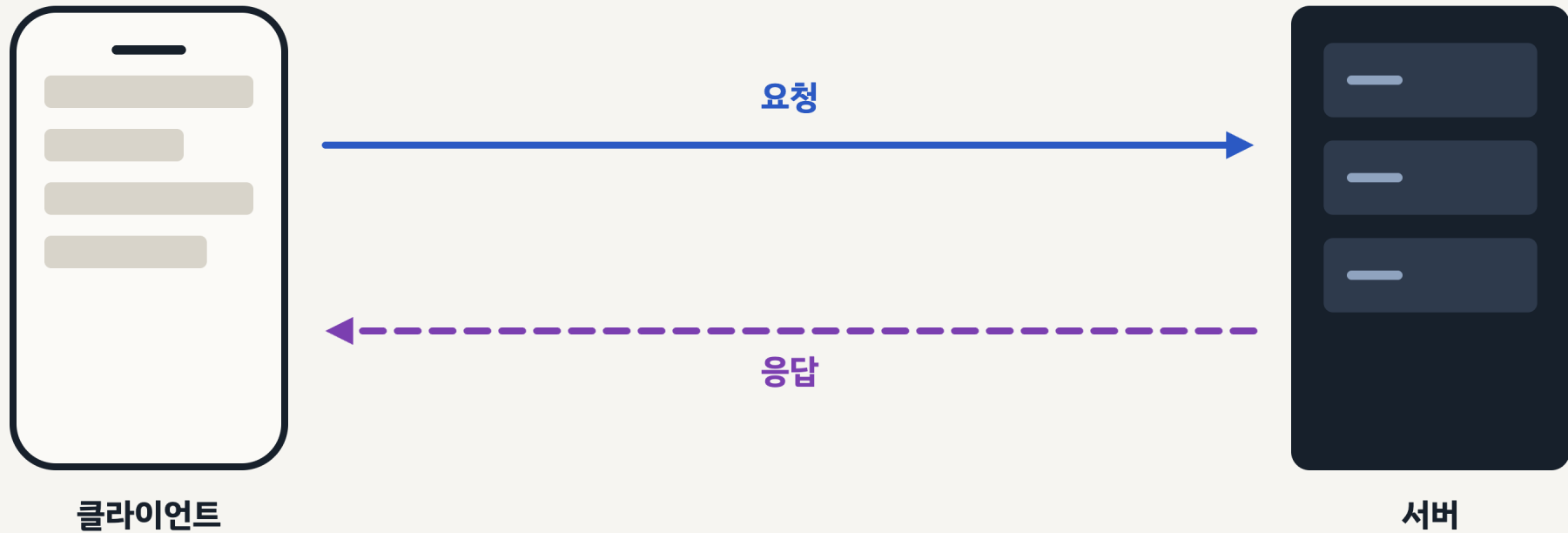


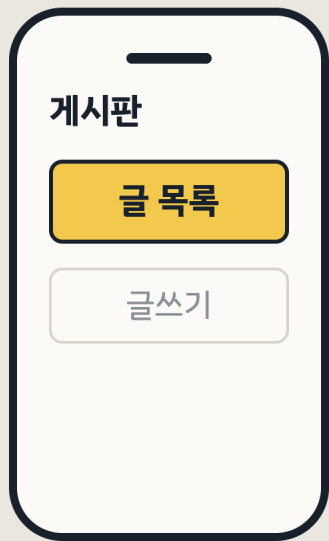
HTTP와 API

백엔드 기초 세미나 1회차

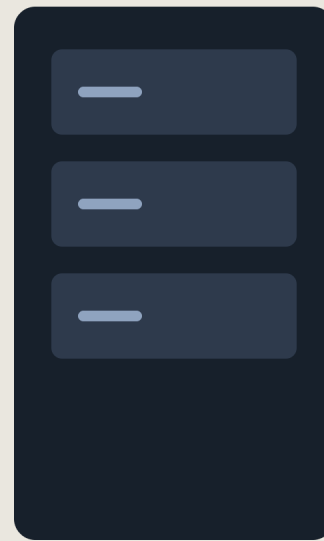


오늘의 질문

게시판 앱에서 '글 목록'을 누르면 무슨 일이 일어날까



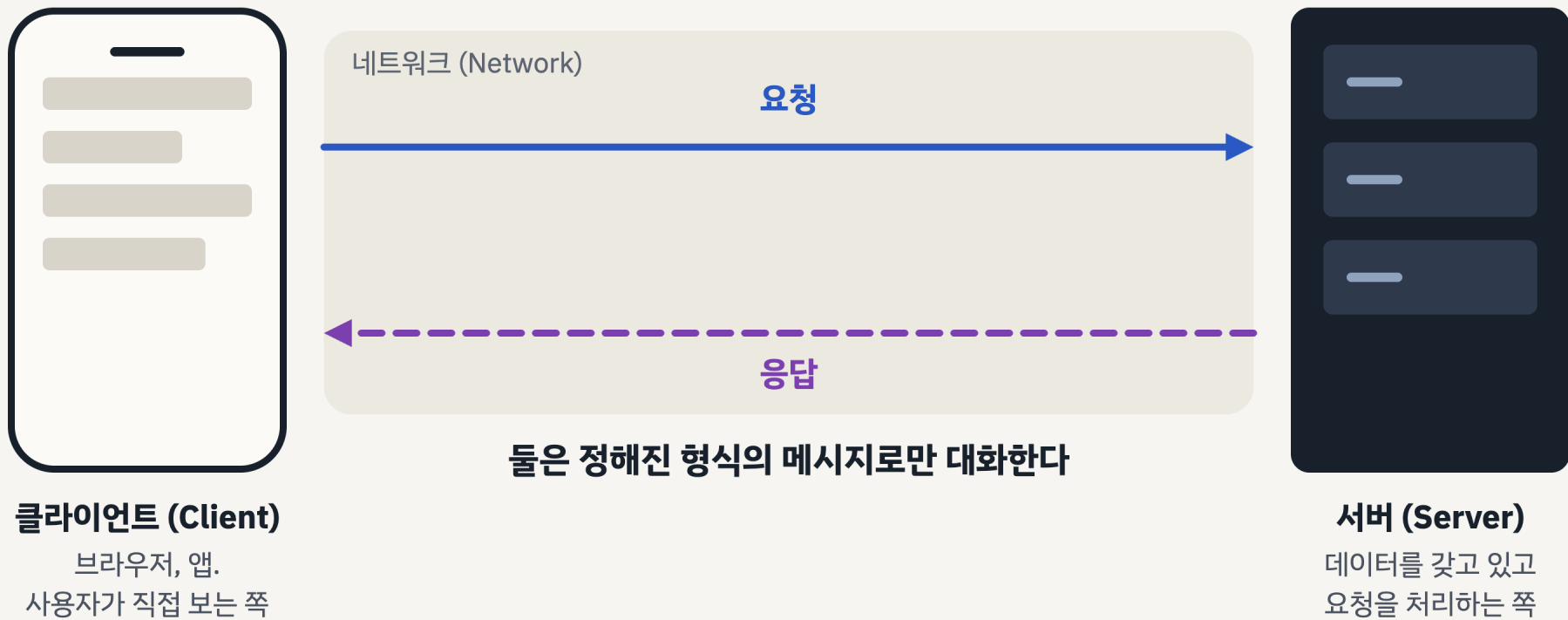
클라이언트



서버

1부 · 백엔드가 하는 일

클라이언트와 서버



1부 · 백엔드가 하는 일

백엔드가 맡는 일

01

글을 저장한다

쓴 글이 사라지지 않게 보관한다

02

규칙을 지킨다

제목이 비어 있으면 거절
목록은 최신순

03

권한을 확인한다

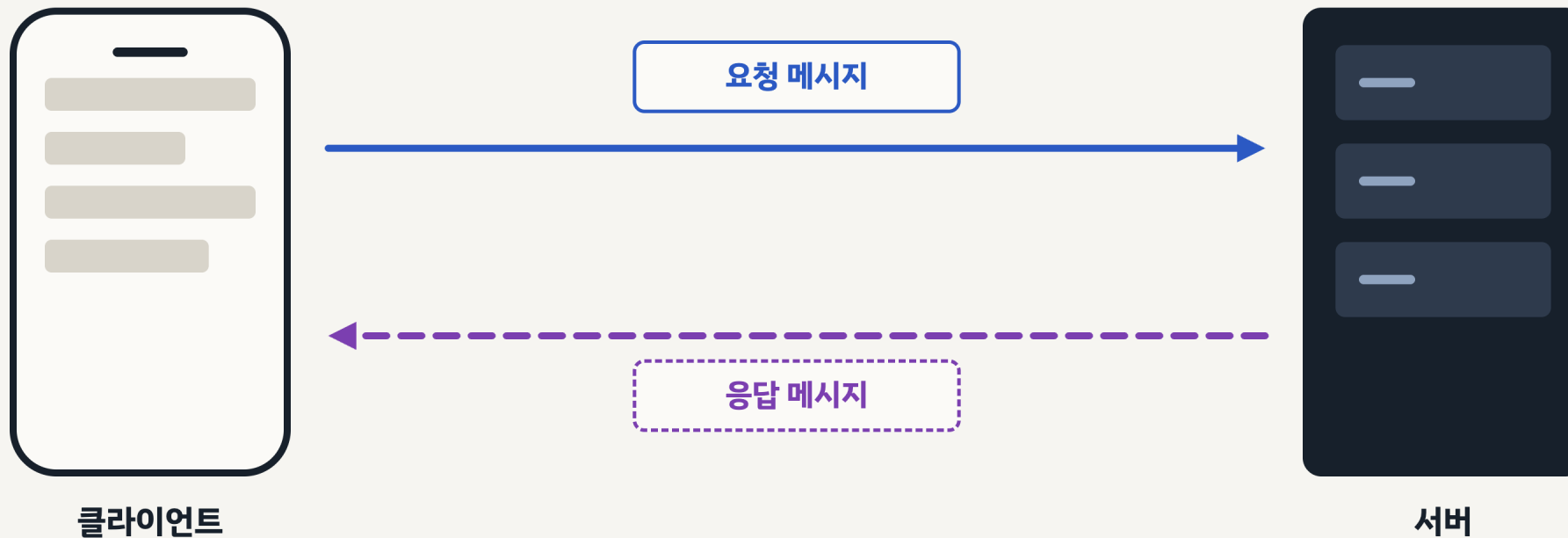
글은 쓴 사람만 지울 수 있다

HTTP

클라이언트와 서버가 주고받는 메시지의 형식

요청 하나에 응답 하나

사람이 읽을 수 있는 텍스트



요청 메시지 (Request)

```
POST /posts HTTP/1.1
```

```
Host: board.example.com
```

```
Content-Type: application/json
```

```
Authorization: Bearer eyJhbGciOi...
```

```
{"title": "첫 글", "content": "안녕하세요"}
```

시작줄 (Start line)

메서드, 경로, 버전

헤더 (Header)

부가 정보

빈 줄 하나가 헤더와 본문을 가른다

본문 (Body)

실제로 보내는 데이터

응답 메시지 (Response)

HTTP/1.1 201 Created

Content-Type: application/json

Location: /posts/42

{"id": 42, "title": "첫 글", "content": "안녕하세요"}

시작줄

버전, 상태코드, 설명

헤더

부가 정보

빈 줄

본문

결과 데이터

본문의 형식, JSON

데이터를 텍스트로 적는 약속. 요즘 API 본문은 대부분 이 형식이다

```
{  
  "id": 3,  
  "title": "첫 글",  
  "notice": false,  
  "tags": ["질문", "HTTP"],  
  "editedAt": null  
}
```

{ }

객체 (object)

이름과 값의 묶음. "title": "첫 글"

[]

배열 (array)

여러 개를 순서대로. 글 목록도 배열이다

값

값의 종류

문자열은 큰따옴표로 감싼다

숫자, true / false, null은 그대로 쓴다

Content-Type: application/json

본문이 JSON이라고 알려 주는 헤더

메모

JavaScript 객체보다 엄격하다. 키에 큰따옴표 필수, 마지막 쉼표와 주석은 쓸 수 없다.

메서드 (Method)

메서드	뜻	게시판에서
GET	조회	글 목록 보기, 글 하나 보기
POST	생성	새 글 쓰기
PUT	전체 교체	글을 통째로 바꾸기
PATCH	일부 수정	제목만 고치기
DELETE	삭제	글 지우기

같은 요청을 두 번 보내면

메서드

서버 상태를 바꾸지 않음 (safe)

여러 번 보내도 결과가 같음 (idempotent)

GET	예	예
POST	아니오	아니오
PUT	아니오	예
PATCH	아니오	보장 안 됨
DELETE	아니오	예

메모 좋아요 수를 1 올리는 PATCH는 보낼 때마다 결과가 달라진다.

상태코드 (Status Code)

2xx

성공

3xx

다른 주소로 이동

4xx

요청이 잘못됨

5xx

서버가 실패함

자주 보는 상태코드

코드	뜻	게시판에서
200 OK	성공	글 목록 조회
201 Created	새로 만들어짐	글 작성
204 No Content	성공, 돌려줄 내용 없음	글 삭제
400 Bad Request	요청 형식이 잘못됨	제목 없이 글쓰기
401 Unauthorized	누구인지 알 수 없음	로그인 없이 글쓰기
403 Forbidden	누구인지는 알지만 허락 안 됨	남의 글 삭제
404 Not Found	대상이 없음	없는 글 번호 조회
500 Internal Server Error	서버 내부 오류	서버 코드 버그

메모 401은 신분증이 없는 경우, 403은 신분증은 있지만 출입 명단에 없는 경우.

헤더 (Header)

헤더	하는 일	예
Content-Type	본문의 형식을 알린다	application/json
Authorization	보낸 사람이 누구인지 증명한다	Bearer eyJhbGciOi...
Cookie	서버가 전에 맡긴 값을 돌려준다	session=abc123

서버는 이전 요청을 기억하지 않는다

클라이언트

로그인할게요

글 쓸게요

서버

확인했습니다

누구시죠?

- 요청 하나하나가 서로 독립적이다 (stateless)
- 그래서 요청마다 신원 정보를 같이 보낸다 (쿠키, 토큰)

로그인 상태를 전달하는 두 방식

	세션 (Session) + 쿠키	토큰 (JWT)
로그인하면 받는 것	서버가 기억할 세션 번호 Set-Cookie: session=abc123	서명된 토큰 (사용자 정보 포함) eyJhbGciOi...
이후 요청마다	브라우저가 자동으로 붙인다 Cookie: session=abc123	클라이언트가 직접 붙인다 Authorization: Bearer eyJ...
서버가 확인하는 법	저장해 둔 세션 목록에서 찾는다	서명이 맞는지 계산해 본다
강제 로그아웃	세션을 지우면 바로 끝난다	만료 전에는 막기 어렵다

쿠키는 값을 실어 나르는 통이고, 세션과 JWT는 무엇을 싣느냐의 차이다

메모 JWT 내용은 암호화가 아니라 인코딩이라 누구나 읽을 수 있다. 비밀번호 같은 값은 넣지 않는다.

직접 열어 보기

1. 아무 사이트에서 F12, Network 탭
2. 새로고침
3. 요청 하나를 클릭해서 메서드, 상태코드, 헤더, 응답 본문 확인

Elements	Console	Network
Name	Method	Status
posts?page=1	GET	200
main.js	GET	200
style.css	GET	200
Headers		
① Request Method: GET		
② Status Code: 200 OK		
③ Content-Type: application/json		

API

이 주소로 이렇게 요청하면 이렇게 응답한다는 약속

프론트엔드와 백엔드는 이 약속만 보고
각자 개발한다

GET	/posts	글 목록
POST	/posts	새 글 쓰기
GET	/posts/3	3번 글 보기
PATCH	/posts/3	3번 글 일부 수정
DELETE	/posts/3	3번 글 삭제

주소에는 대상, 행동은 메서드로

피할 것

쓸 것

GET /getPosts

→

GET /posts

POST /createPost

→

POST /posts

POST /updatePost?id=3

→

PATCH /posts/3

GET /deletePost?id=3

→

DELETE /posts/3

주소 짓기

/posts

글 전체

└ **/posts/3**

3번 글

└ **/posts/3/comments**

3번 글의 댓글

복수형 명사

/posts

/post 가 아니라

소문자

/posts

/Posts 가 아니라

단어 사이는 하이픈

/hot-posts

/hotPosts, /hot_posts 가 아니라

경로 파라미터와 쿼리 파라미터

경로 파라미터 (Path parameter)

대상 하나를 고를 때

/posts/3

쿼리 파라미터 (Query parameter)

목록을 거르거나 정렬할 때

/posts?page=2&sort=latest

https://board.example.com/posts/3?page=2

서버 주소

경로

쿼리

응답 모양 맞추기

성공 **200 OK** GET /posts/3

```
{
  "id": 3,
  "title": "첫 글",
  "content": "안녕하세요"
}
```

실패 **404 Not Found** GET /posts/999

```
{
  "error": {
    "code": "POST_NOT_FOUND",
    "message": "글을 찾을 수 없습니다"
  }
}
```

성공이든 실패든 정해진 모양으로 돌려준다

메모 실패를 200으로 보내면 상태코드를 보는 브라우저, 모니터링, 재시도 로직이 성공으로 착각한다.

API 명세 (Specification)

주소, 메서드, 요청 형식, 응답 형식,
에러를 정리한 문서

많이 쓰는 표준:

OpenAPI (Swagger)

게시판 API

OpenAPI 3.0

posts

GET

/posts

글 목록

POST

/posts

새 글 쓰기

GET

/posts/{id}

글 하나 보기

PATCH

/posts/{id}

글 일부 수정

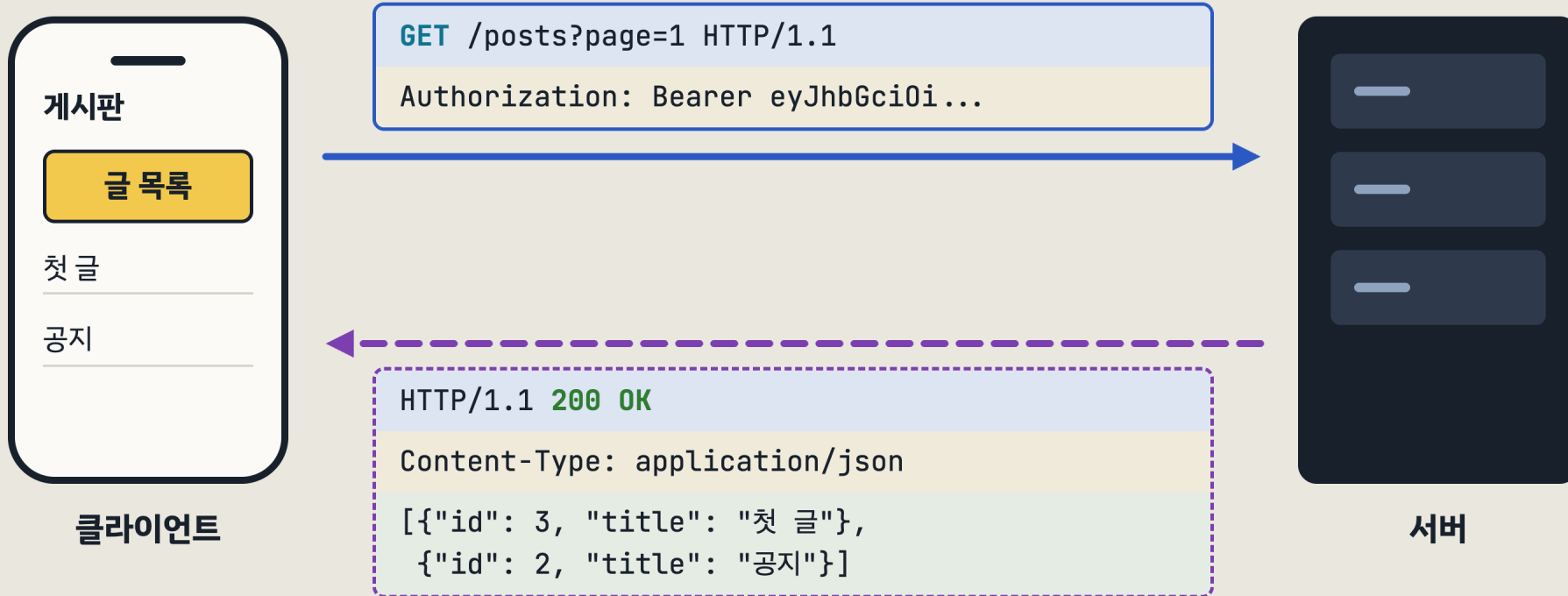
DELETE

/posts/{id}

글 삭제

마무리

다시, '글 목록' 버튼



부록

자주 나오는 질문

PUT과 PATCH

PUT은 보낸 내용으로 통째로 교체한다.

빠뜨린 필드는 사라질 수 있다.

PATCH는 보낸 필드만 고친다.

CORS

브라우저가 다른 출처 서버의 응답을 막는 보안 규칙.

프론트엔드를 처음 서버에 붙일 때 거의 반드시 만난다.

HTTP/2, HTTP/3

메서드·상태코드·헤더의 의미는 그대로이고

전송 방식만 빨라졌다.

HTTPS

같은 HTTP 메시지를 암호화된 연결로 주고받는다.

내용은 같고 중간에서 엿보지 못하게 할 뿐이다.